AresGuard: Mission Control System

Distributed Automation Platform for Mars Habitat

Ettore Cantile 2026562
Leonardo Chiarparin 2016363

Laboratory of Advanced Programming

(Hackaton: March 2026)

MISSION BRIEFING

Scenario overview: Mars Habitat 2036

THE CONTEXT

HOSTILE ENVIRONMENT

The year is 2036. You have been deployed to Mars Operations at SpaceY. The habitat's automation stack is partially destroyed and unstable.Life-support relies on manual polling of heterogeneous sensors. Without intervention, the crew faces a thermodynamic collapse.

THE CHALLENGE

SYSTEM FRAGMENTATION

Sensors communicate using incompatible dialects (rest.scalar, rest.chemistry). Critical data is fragmented and requires continuous manual polling. There is no unified visualization for vital signs. Automated safety protocols are offline or non-existent.

THE OBJECTIVE

SOLUTION: ARESGUARD

Design and deploy a distributed automation platform. Ingest data from all legacy REST devices. Normalize payloads into a Unified Event Schema. Automate critical responses via a persistent Rule Engine. Visualize real-time status on a centralized Mission Control Dashboard.

PROJECT SCOPE & CORE ACHIEVEMENTS

2-PERSON TEAM STRATEGY

- **Configuration**: Optimized for a 2-Developer Team
- **Ingestion Approach**: Robust REST Polling (5s interval)

  Note: *Telemetry streaming omitted as per mission guidelines.*

- ***Primary Goal****: Maximize system resilience and data consistency over complexity*
- ***Validation****: 10 Core User Stories fully implemented*

DELIVERED ARCHITECTURE

- **Microservices**: Fully containerized stack via Docker Compose
- **Event-Driven:** Asynchronous processing via RabbitMQ
- **Automation**: Persistent Rule Engine (PostgreSQL) with Upsert logic
- **Visualization**: Real-time Dashboard with WebSocket push
- **IaC**: Infrastructure defined strictly as Code

EVENT-DRIVEN ARCHITECTURE & DATA FLOW

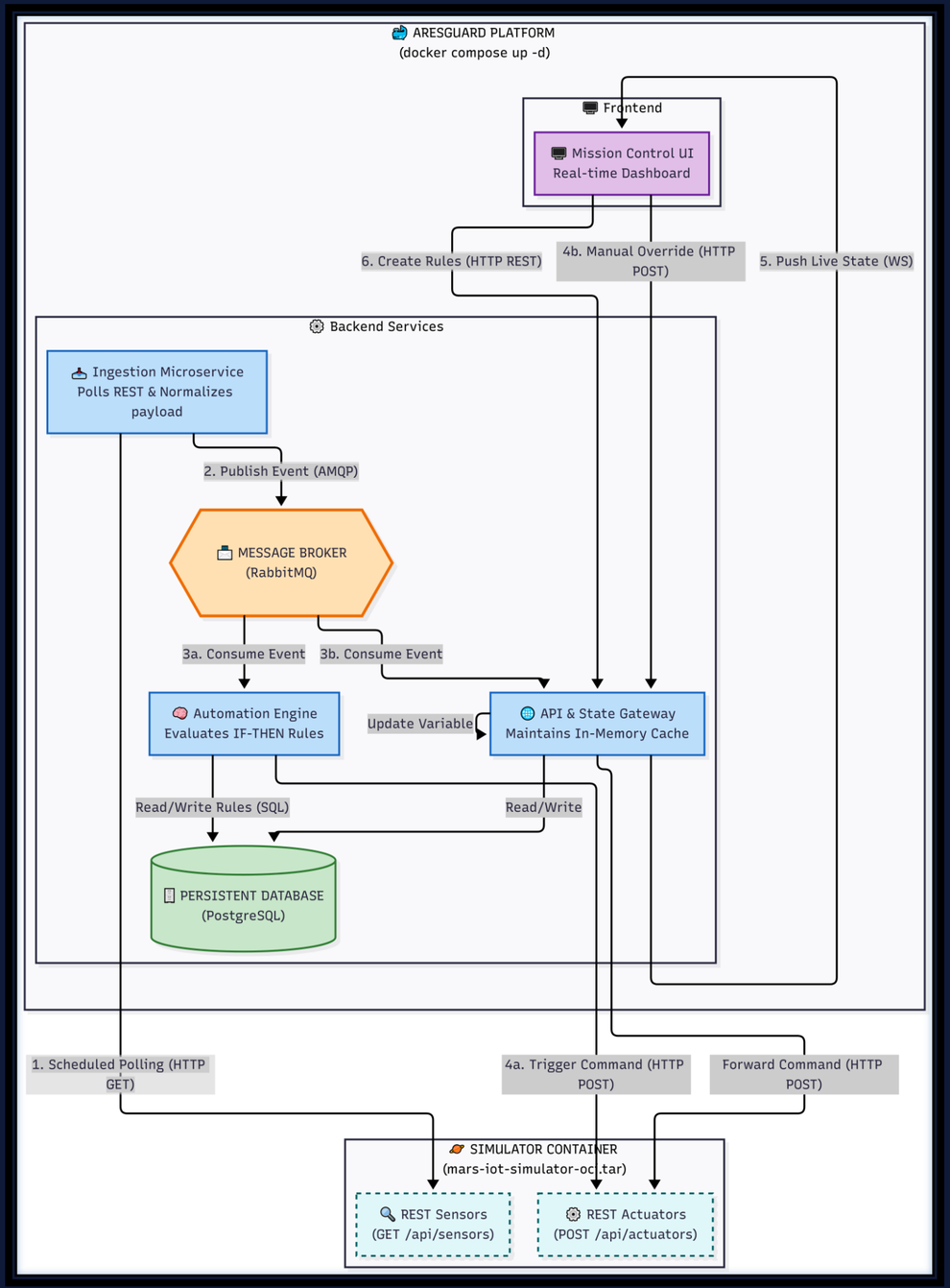
High-Level Overview:

1. **Decoupled Ingestion Layer**: A dedicated microservice polls legacy REST sensors (5s interval), normalizing heterogeneous data into a Unified Event Schema. It publishes atomic events to RabbitMQ, ensuring data intake never blocks processing.

2. **The "Fan-out" Backbone**: The Message Broker broadcasts every event simultaneously to all consumers. This architecture decouples the "source" from the "brains," guaranteeing system resilience even if a component fails.

3. **Persistent Automation**: The Rule Engine consumes events to evaluate safety conditions dynamically. All rules are stored in PostgreSQL, ensuring critical safety protocols survive system reboots.

4. **Real-Time Visualization**: The API Gateway updates the system state in-memory and pushes live telemetry to the Dashboard via **WebSockets**, providing zero-latency situational awareness.



FROM RAW SENSORS TO UNIFIED EVENTS

The Normalization Process

Legacy Polling

The Ingestion Microservice queries the Simulator API every 5 seconds (REST GET). Target: Heterogeneous sensors with incompatible dialects (e.g., rest.scalar vs rest.chemistry).

Protocol Decoding

The service actively identifies the sensor type. Complex nested payloads (like chemistry arrays) are unpacked and flattened into individual metric readings.

Unified Schema

All data is converted into a standard JSON format:

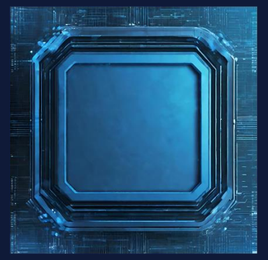
- **source**: ID & Protocol
- **payload**: Value & Unit
- **timestamp**: ISO-8601

Broker Dispatch

The sanitized, atomic event is published to RabbitMQ (ares_telemetry_stream).The ingestion process ends here, fully decoupled from the rest of the system.

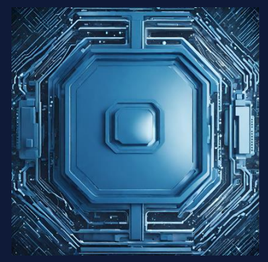
THE NERVOUS SYSTEM: RABBITMQ BACKBONE

The Nervous System: RabbitMQ Backbone



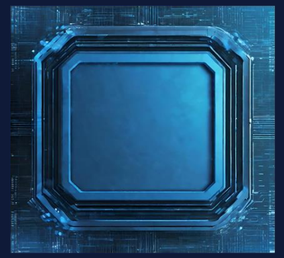
Fire & Forget Ingestion

The Ingestion Microservice publishes normalized events to the exchange. It operates blindly, unaware of who is listening, maximizing throughput and decoupling data intake from processing logic.



Fan-out Exchange

RabbitMQ sits at the center, acting as the distribution hub. We utilize a Fan-out pattern to instantaneously broadcast every single telemetry event to multiple queues simultaneously, ensuring parallel processing.



Parallel Execution

Two distinct services consume the same message stream in parallel:

- **Automation Engine**
- **API Gateway**

***Note**: If one fails, the other continues uninterrupted.*

THE BRAIN: PERSISTENT RULE ENGINE

AresGuard implements a deterministic logic engine that consumes telemetry in real-time to enforce safety protocols.

- **Dynamic Evaluation**: Simple **IF-THEN** logic triggers actuators instantly when thresholds are breached. Syntax: **IF** *<sensor>* *<operator>* *<value>* **THEN** *<actuator>* = **ON**/**OFF**
- **Resilient Persistence**: Rules are not just in RAM. They are stored in PostgreSQL, ensuring safety protocols survive system reboots or crashes
- **Smart Conflict Resolution**: A database-level constraint ***UNIQUE****(sensor, actuator, operator)* prevents logic collisions. The system uses **UPSERT** to automatically update existing rules instead of creating duplicates

MISSION CONTROL INTERFACE

Real-Time Situational Awareness

Zero-Latency Updates

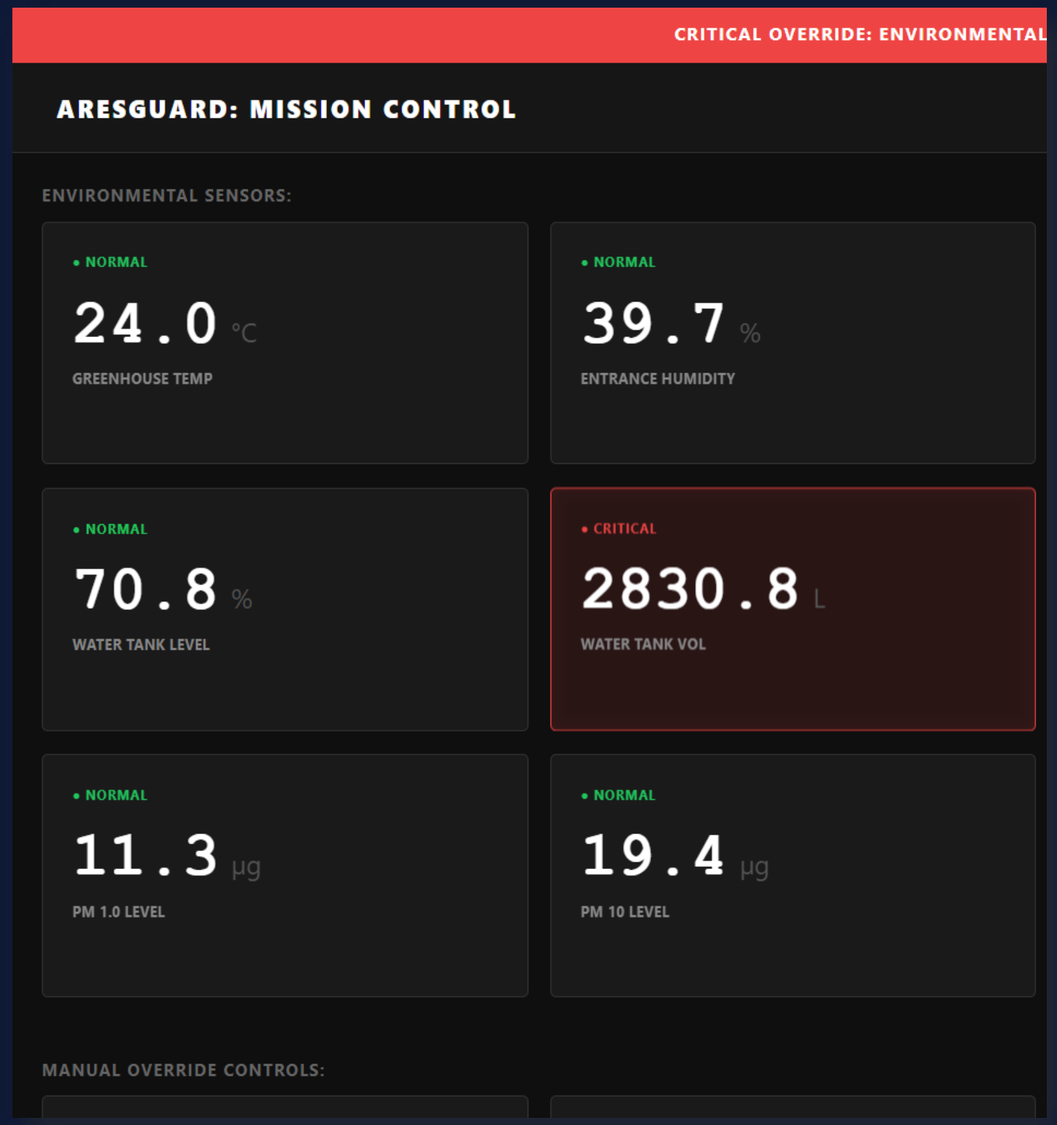
The API Gateway pushes telemetry via WebSocket (/ws) directly to the browser. Immediate state synchronization without manual page reloads (Push vs Pull).

Lightweight Tech Stack

Pure Vanilla JS: Built with native HTML5, CSS3, and JavaScript. No heavy frameworks ensures maximum performance on constrained hardware.

Astronaut-Centric UX

- Dark Mode: High-contrast UI designed for low-light habitat environments
- Visual Alarms: Sensor cards flash RED instantly when safety thresholds are breached
- Interactive Data: Integrated Chart.js renders historical trends on-demand for deep analysis



DEPLOYMENT & ORCHESTRATION

```
PROBLEMS    OUTPUT    TERMINAL    PORTS

PS C:\Users\ecant\.vscode\2026562_PROJECT\source> docker compose up -d
[+] up 8/8
 ✓ Network source_aresguard_net    Created
 ✓ Container aresguard_db          Healthy
 ✓ Container mars_simulator        Created
 ✓ Container aresguard_broker      Healthy
 ✓ Container aresguard_ingestion   Created
 ✓ Container aresguard_rule_engine Created
 ✓ Container aresguard_api_gateway Created
 ✓ Container aresguard_frontend    Created
PS C:\Users\ecant\.vscode\2026562_PROJECT\source>
```

Containerized Stack

The entire mission ecosystem (Ingestion, Broker, DB, Frontend) is defined in a single docker-compose.yml file. No manual installation is required.

12 Factor Portability

We strictly use Environment Variables (os.getenv) for configuration. The system is agnostic: it runs identically on a developer laptop or a cloud server without code changes.

ENGINEERING CHALLENGES & SOLUTIONS

Beyond the Basic Requirements

LOGIC CONFLICTS

- **The Problem**: What if a user defines two contradictory rules for the same sensor (e.g., > 20 and > 25)?
- **The Solution**: We implemented a strict Database Constraint UNIQUE(sensor, actuator, operator). The backend uses UPSERT logic: new rules automatically update existing ones instead of creating ambiguous duplicates

INPUT VALIDATION

- **The Problem**: Preventing "Garbage In, Garbage Out" (e.g., setting a temperature threshold of -500°C)
- **The Solution**: The UI enforces physical limits based on sensor types before sending data to the API. *Result: Invalid commands are rejected at the source, reducing backend load*

OBSERVABILITY

- **The Problem**: Debugging a distributed asynchronous system is difficult when events happen in milliseconds
- **The Solution**: We built a custom "Expand View" in the dashboard log. It allows operators to inspect the full JSON payload of every automated action without cluttering the main UI

CONCLUSION

Project Summary & Compliance

MISSION STATUS: READY FOR LAUNCH

1. **Architecture Compliance**
   - **Microservices:** Fully containerized, decoupled architecture (Ingestion, Broker, Backend, Frontend)
   - **Event-Driven**: Asynchronous communication via RabbitMQ (Fan-out pattern)
2. **Functional Requirements**
   - **Data Ingestion**: Robust polling and normalization of legacy sensors
   - **Automation**: Persistent Rule Engine with database integrity (UNIQUE constraints)
   - **Visualization**: Real-time, zero-latency dashboard via WebSockets
3. **Development Standards**
   - **Infrastructure as Code**: Single docker-compose.yml for reproducible deployment
   - **Clean Code**: Modular Python backend and optimized Vanilla JS frontend

>> INITIATING LIVE DEMO SEQUENCE…

MISSION ACCOMPLISHED

Thank you for your attention!